

Flutter

1. Does Dart support inheritance?

Yes. Dart supports **single inheritance** using the `extends` keyword.

A class can extend **only one parent class**.

Basic example

```
class Animal {  
  void eat() {  
    print("Animal is eating");  
  }  
}  
  
class Dog extends Animal {  
  void bark() {  
    print("Dog is barking");  
  }  
}  
  
void main() {  
  Dog d = Dog();  
  
  d.eat();    // inherited from Animal  
  d.bark();   // Dog's own method  
}
```

Output:

```
Animal is eating  
Dog is barking
```

Important exam point

```
class Dog extends Animal
```

means:

`Dog` is a child/subclass of `Animal` and gets access to the non-private members of `Animal`.

2. Does Dart support multilevel inheritance?

YES.

Multilevel inheritance means:

```
Grandparent
  ↓
Parent
  ↓
Child
```

Example

```
class Animal {
  void eat() {
    print("Eating");
  }
}

class Dog extends Animal {
  void bark() {
    print("Barking");
  }
}

class Puppy extends Dog {
  void play() {
```

```

        print("Playing");
    }
}

void main() {
    Puppy p = Puppy();

    p.eat();    // Animal
    p.bark();   // Dog
    p.play();   // Puppy
}

```

Here:

```

Animal
  ↓
  Dog
  ↓
  Puppy

```

`Puppy` can access methods from **both** `Dog` and `Animal`.

Remember:

```

Puppy p = Puppy();

p.eat();    // inherited indirectly from Animal
p.bark();   // inherited from Dog
p.play();   // its own method

```

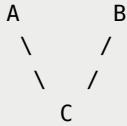
3. What about multiple inheritance?

Dart **does NOT** support multiple class inheritance.

You cannot do:

```
class C extends A, B { } // ❌ ERROR
```

A class can extend only **one class**.



❌ Not allowed with classes in Dart

But Dart provides **mixins** to achieve similar code-reuse behavior.

4. What is a Mixin?

A **mixin** is a way to reuse methods/properties in multiple classes **without using inheritance**.

You create one using:

```
mixin Logger {  
  void log() {  
    print("Logging...");  
  }  
}
```

Then use it with:

```
class User with Logger {  
  void login() {  
    print("User logged in");  
  }  
}
```

Now:

```
void main() {  
    User u = User();  
  
    u.login();  
    u.log();  
}
```

Output:

```
User logged in  
Logging...
```

5. Why do we need mixins?

Suppose we have:

```
class Flyable {  
    void fly() {  
        print("Flying");  
    }  
}
```

Both birds and airplanes can fly, but they don't necessarily have an **is-a inheritance relationship**.

Instead:

```
mixin Flyable {  
    void fly() {  
        print("Flying");  
    }  
}  
  
class Bird with Flyable {
```

```
}  
  
class Airplane with Flyable {  
}
```

Now both can reuse `fly()`:

```
Bird b = Bird();  
Airplane a = Airplane();  
  
b.fly();  
a.fly();
```

This is **code reuse through mixins**.

6. `extends` vs `with`

This is VERY important for exams.

`extends`

Used for **inheritance**:

```
class Dog extends Animal {  
}
```

Meaning:

| Dog **is an** Animal.

`with`

Used for **mixin**:

```
class Dog with Swimmable {  
}
```

Meaning:

| Dog gets/reuses the functionality of `Swimmable`.

7. Can we use inheritance + mixin together?

YES.

```
class Animal {
  void eat() {
    print("Eating");
  }
}

mixin Flyable {
  void fly() {
    print("Flying");
  }
}

class Bird extends Animal with Flyable {
  void chirp() {
    print("Chirping");
  }
}

void main() {
  Bird b = Bird();

  b.eat();    // Animal
  b.fly();    // Flyable
  b.chirp();  // Bird
}
```

Structure:

```
Animal
  ↑
  |
Bird ← Flyable
```

Bird:

- **inherits** from `Animal`
- **uses** `Flyable` mixin
- has its own `chirp()` method

8. Can a class use multiple mixins?

YES.

This is one of the major advantages of mixins.

```
mixin Flyable {
  void fly() {
    print("Flying");
  }
}

mixin Swimmable {
  void swim() {
    print("Swimming");
  }
}

class Duck with Flyable, Swimmable {
}
```

Now:

```
void main() {
    Duck d = Duck();

    d.fly();
    d.swim();
}
```

Output:

```
Flying
Swimming
```

So:

```
class Duck with Flyable,Swimmable
```

is valid.

9. Inheritance vs Mixin - exam table

Feature	Inheritance	Mixin
Keyword	<code>extends</code>	<code>with</code>
Purpose	Establish parent-child relationship	Reuse functionality
Multiple classes?	✗ No	✓ Multiple mixins
Example	<code>Dog extends Animal</code>	<code>Dog with Flyable</code>
"is-a" relationship	✓	Not necessarily
Multilevel	✓ Supported	N/A

10. One exam question you should expect

Q: Does Dart support multiple inheritance?

Answer:

No, Dart does not support multiple inheritance of classes. A class can extend only one class. However, Dart supports mixins, which allow a class to reuse functionality from multiple mixins using the `with` keyword.

Example:

```
mixin A {  
  void methodA() {  
    print("A");  
  }  
}  
  
mixin B {  
  void methodB() {  
    print("B");  
  }  
}  
  
class C with A, B {  
}
```

11. Quick memory trick

Remember these three:

`extends` → inheritance → ONE parent class

`with` → mixins → MULTIPLE mixins

multilevel → YES

So for your exam:

```
class B extends A      //  
class C extends B      // multilevel
```

```
class C extends A, B      // multiple inheritance
```

```
class C with A, B        // multiple mixins
```

Package vs Plugin - one perfect exam answer

If your teacher asks:

"What is the difference between a package and a plugin in Flutter?"

Say:

A **package** is reusable Dart or Flutter code that can be added as a dependency to an application. A **plugin** is a specialized type of package that exposes functionality involving platform-specific APIs, such as camera, GPS, Bluetooth, or sensors. Packages generally contain reusable Dart/Flutter code, while plugins may contain both Dart code and native platform implementations.

What is a Package?

A **Dart package** is reusable Dart code that can be added to another Dart/Flutter project.

It usually contains:

```
lib/  
  package_code.dart  
pubspec.yaml
```

Examples of things a package can provide:

- Utility functions
- Models
- Business logic

- UI widgets
- State-management code
- Algorithms

Example:

```
lib/  
  package_code.dart  
pubspec.yaml
```

Another Flutter app can use it:

```
import 'package:my_package/calculator.dart';  
void main() {  
  print(add(2,3));  
}
```

Key idea

Package = reusable Dart/Flutter code.

What is a Plugin?

A **Flutter plugin** is a special type of package that provides functionality that interacts with the **native platform**.

For example:

```
Flutter/Dart  
  ↓  
  Plugin  
  ↓  
Android / iOS / Web / Windows etc.
```

Examples:

- Camera

- GPS/location
- Bluetooth
- Sensors
- Device information
- Native notifications

A plugin can contain platform-specific code such as:

```
Dart
↓
Android → Kotlin/Java
iOS      → Swift/Objective-C
```

For example, a location plugin lets your Dart code ask the phone:

```
final location=await getLocation();
```

while the plugin handles the native Android/iOS implementation underneath.

Package vs Plugin

This is the important comparison:

	Package	Plugin
Main purpose	Reusable code	Platform-specific functionality
Dart code	✓	✓
Native code required	✗ Usually	✓ Often
Can contain widgets?	✓	✓
Can access native APIs?	Not directly	✓
Can be used by Flutter app?	✓	✓
<code>pubspec.yaml</code>	✓	✓

Easy way to remember

Package:

| "Here is some reusable code."

Plugin:

| "Here is reusable code that can communicate with the platform."

Can a Flutter App be packaged?

This question has **two meanings**, so be careful in the exam.

Meaning 1 - Can an app be distributed as a package?

A normal Flutter **application project is not a package but yes it can be packaged**.

An app has an entry point:

```
voidmain() {runApp(MyApp());  
}
```

and produces an executable application:

```
Android → APK/AAB  
iOS     → IPA  
Web     → Web application  
Windows → executable
```

So you generally don't convert an entire Flutter app into a package simply by changing its name.

But...

You **can extract reusable parts of an app into a package**.

For example:

```
My Flutter App  
|
```

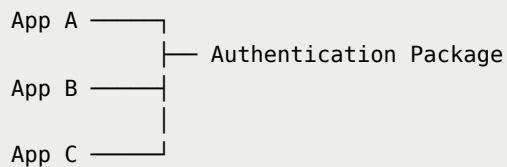
```
├ Screens
├ Widgets
├ Models
├ API logic
└ Authentication
```

You might extract:

```
authentication/  
  lib/  
    auth.dart
```

and turn that functionality into a reusable package.

Then:



So:

App → can have reusable code extracted into a package.

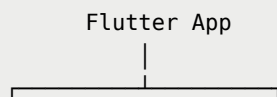
But the **app itself is not normally a package.**

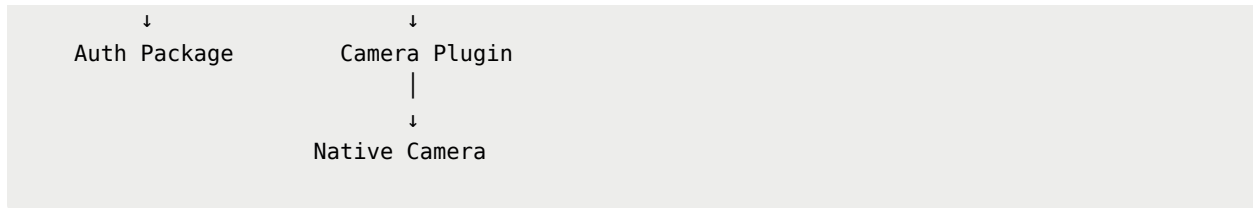
Can a Plugin be an App?

Not normally.

A plugin is designed to be **used by an app**, not to be the standalone application itself.

Think:





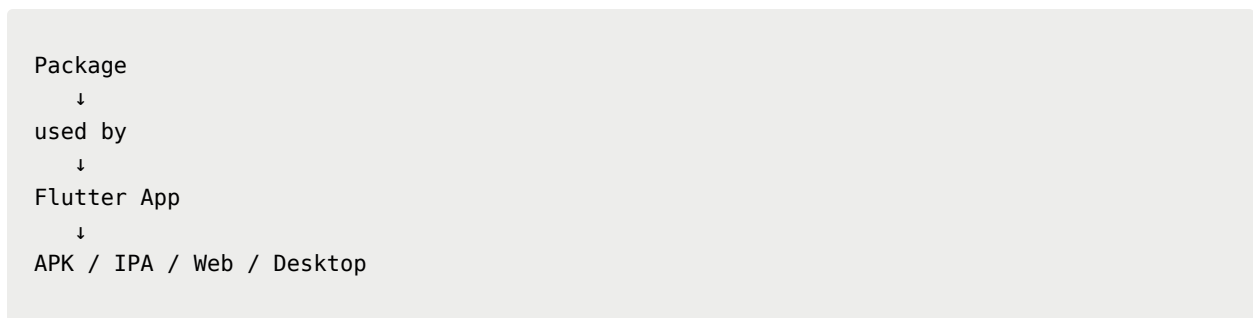
The plugin doesn't normally provide the application's UI/entry point.

Can a Package be an App?

Again, **not directly**.

A package is a reusable library.

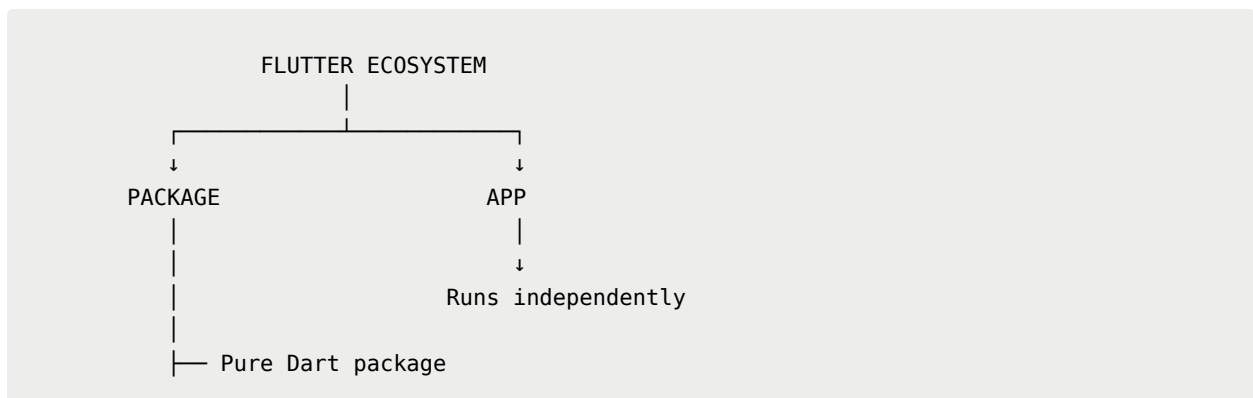
An application is the final product that runs.

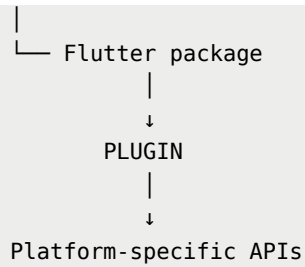


However, Flutter tooling has different project templates, and you can create a project containing reusable code and then create an application that consumes it.

Very Important: Package vs Plugin vs App

Think of them like this:





A plugin is essentially a **specialized package**.

Types of Packages

You may hear these terms:

Dart package

Pure Dart code.

```
my_package/  
├─ lib/  
├─ test/  
└─ pubspec.yaml
```

Example:

```
class Calculator {  
  int add(int a,int b)=>a+b;  
}
```

No Flutter UI or native platform code is necessary.

Flutter package

A package specifically designed for Flutter.

It can contain:

```
Widget myWidget() {  
  return Text("Hello");  
}
```

```
}
```

It can use:

```
import'package:flutter/material.dart';
```

So:

Flutter package = reusable Flutter/Dart code.

Flutter plugin

A package that provides a platform-specific API.

For example:

```
camera_plugin
|
├─ Dart API
├─ Android implementation
├─ iOS implementation
└─ ...
```

What is a Dependency?

Another related concept your professor may ask.

Suppose your app uses:

```
http
provider
camera
```

These are **dependencies**.

You specify them in:

```
dependencies:
  flutter:
    sdk: flutter

  http: ^1.0.0
  provider: ^6.0.0
```

Then Flutter gets those packages from **pub.dev** or another package source.

So:

```
Your App
├── http package
├── provider package
└── camera plugin
```

What is **pub.dev** ?

pub.dev is the package repository/ecosystem for Dart and Flutter.

Developers publish packages/plugins there.

For example:

```
pub.dev
├── Packages
├── Plugins
└── Libraries
```

Your **pubspec.yaml** tells Flutter what dependencies your project needs.

What is **pubspec.yaml** ?

Every Dart/Flutter package and Flutter application has a **pubspec.yaml**.

It contains project metadata and dependencies.

Example:

```
name: my_app

dependencies:
  flutter:
    sdk: flutter

http: ^1.2.0
```

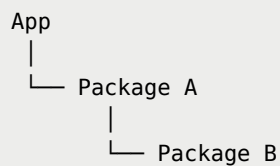
For an app:

```
pubspec.yaml
  ↓
dependencies
  ↓
packages/plugins
```

Can packages depend on packages?

YES.

For example:



Package A can itself depend on Package B.

This is called a **transitive dependency**.

Example:

```
Your App
  ↓
Package A
```

↓
Package B

Your app gets Package B indirectly through Package A.

Can a plugin depend on packages?

YES.

A plugin can have:

- Dart dependencies
- Flutter dependencies
- Platform-specific dependencies

For example:

```
Your App
  ↓
Location Plugin
  ↓
Dart packages
  ↓
Native Android/iOS APIs
```

Can an app contain packages?

An app can **use** packages as dependencies.

It can also have its own internal reusable libraries/code.

For example:

```
my_app/
|
├── lib/
|   ├── screens/
|   ├── widgets/
|   ├── models/
|   └── services/
```

```
|  
├─ test/  
└─ pubspec.yaml
```

Those files are part of the application; they don't automatically become a separately published package.

Package vs Library

This distinction can appear in exams.

Library

A **library** is a unit of Dart code that can be imported.

```
import 'package:my_package/calculator.dart';
```

Package

A package is a **distributable collection/project** containing libraries and other resources.

Think:

```
PACKAGE  
|  
├─ Library A  
├─ Library B  
├─ Library C  
├─ Tests  
└─ pubspec.yaml
```

So:

Library = code unit

Package = distributable collection/project

Dart: Overloading vs Overriding

Your note:

Dart doesn't support traditional method overloading, but it supports method overriding.

What is Method Overloading?

Overloading means having multiple methods with the **same name but different parameter lists** in the same class.

For example, in Java:

```
class Calculator {  
  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

Same method name:

```
add()  
add()
```

but different parameters.

That's **method overloading**.

Does Dart support method overloading?

No - not traditional/static method overloading.

You cannot do:

```
class Calculator {
  int add(int a, int b) {
    return a + b;
  }

  int add(int a, int b, int c) { // ❌
    return a + b + c;
  }
}
```

Dart will give an error because you cannot define two methods with the same name in the same class.

Why?

Dart doesn't select methods based on different parameter signatures in the same way languages such as Java/C++ do.

But Dart can achieve similar behavior

Dart provides features such as:

Optional positional parameters

```
class Calculator {
  int add(int a, int b, [int? c]) {
    return a + b + (c ?? 0);
  }
}
```

Now:

```
print(Calculator().add(2, 3));
print(Calculator().add(2, 3, 4));
```

Output:

```
5  
9
```

This is **not technically method overloading**.

It's **one method with an optional parameter**.

Named parameters can also help

```
class User {  
  void createUser(String name, {int age = 0}) {  
    print("$name, $age");  
  }  
}
```

You can call:

```
createUser("Ali");
```

or:

```
createUser("Ali",age:20);
```

Again, this isn't traditional method overloading.

What is Method Overriding?

Overriding happens when a **child class provides its own implementation of a method inherited from the parent class**.

Example:

```
class Animal {  
  void sound() {  
    print("Animal makes a sound");  
  }  
}
```

```

    }
}

class Dog extends Animal {
    @override
    void sound() {
        print("Dog barks");
    }
}

```

Now:

```

void main() {
    Dog d = Dog();
    d.sound();
}

```

Output:

```

Dog barks

```

The parent had:

```

sound()

```

and the child **overrode** it with its own implementation.

Why do we use **@override** ?

```

@override
void sound() {
    print("Dog barks");
}

```

`@override` tells Dart:

"I intend to override a method inherited from my parent class."

It's technically an **annotation**, and it helps Dart detect mistakes.

For example:

```
class Dog extends Animal {  
  @override  
  void sounds() { // ❌  
  }  
}
```

If `sounds()` doesn't exist in `Animal`, Dart can flag the incorrect override.

Overloading vs Overriding

	Overloading	Overriding
Where?	Same class	Parent + child
Method name	Same	Same
Parameters	Different	Generally compatible signature
Implementation	Multiple methods	Child replaces parent implementation
Dart supports?	❌ Traditional overloading	Yes
Keyword	None	<code>@override</code>

Another important concept: Constructors

Don't confuse **method overloading** with constructors.

Dart also doesn't support traditional constructor overloading like:

```
Class()  
Class(int x)  
Class(int x, int y)
```

Instead, Dart commonly uses **named constructors**:

```
class Student {  
  String name;  
  int age;  
  
  Student(this.name, this.age);  
  
  Student.fromName(this.name) : age = 0;  
}
```

Then:

```
Student s1 = Student("Ali", 20);  
  
Student s2 = Student.fromName("Ahmed");
```

So remember:

```
Dart  
|  
├─ Traditional method overloading ❌  
├─ Method overriding ✓  
├─ Named constructors ✓  
└─ Optional/named parameters ✓
```

Dart does not support traditional method overloading, where multiple methods have the same name with different parameter lists. However, Dart supports method overriding, where a subclass provides its own implementation of an inherited method, using `@override`.

API vs SDK vs Webview

API = you communicate with the service yourself.

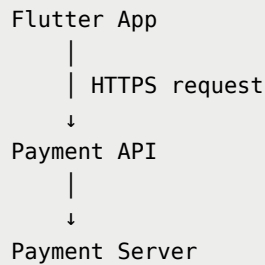
SDK = the provider gives you a ready-made toolkit that communicates with the service for you.

WebView = you open the provider's web-based payment UI inside your app.

API approach

Suppose your Flutter app needs to make a payment using a payment provider.

With an **API**, your application directly sends HTTP requests to the provider.



For example:

```
POST /create-payment
Authorization: API_KEY
Amount: 5000
Currency: PKR
```

Your code handles things like:

- HTTP requests
- Authentication
- Request/response models
- Error handling
- Payment status
- UI
- Security considerations

Example conceptually

```
final response = await http.post(
  Uri.parse("https://payment-provider.com/api/payment"),
  body: {
    "amount": "5000",
    "currency": "PKR",
  },
);
```

Advantage of API

More control.

You decide:

- UI
- request flow
- error handling
- data handling
- architecture
- when/how APIs are called

Disadvantage

More development work.

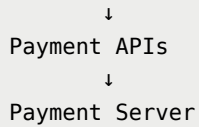
You have to implement much of the integration yourself.

SDK approach

An **SDK (Software Development Kit)** is a collection of tools/libraries provided by the payment provider.

Instead of manually constructing HTTP requests:

```
Your Flutter App
  ↓
Payment SDK
```



The SDK handles a lot of the communication for you.

Conceptually:

```
PaymentSDK.initialize(apiKey);

PaymentSDK.pay(
  amount: 5000,
  currency: "PKR",
);
```

The actual SDK might internally handle:

```
Authentication
API requests
Request formatting
Response parsing
Error handling
Security mechanisms
Platform integration
```

depending on the provider.

Advantage

Less development effort + easier integration.

You use provider-provided methods rather than building the whole integration yourself.

Disadvantage

Less control + dependency on the SDK.

If the SDK:

- doesn't support a feature,

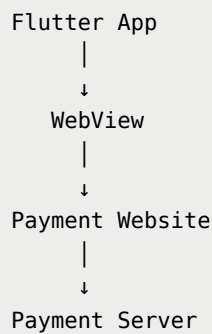
- has a bug,
- becomes outdated,
- isn't available for your platform,

you may have problems.

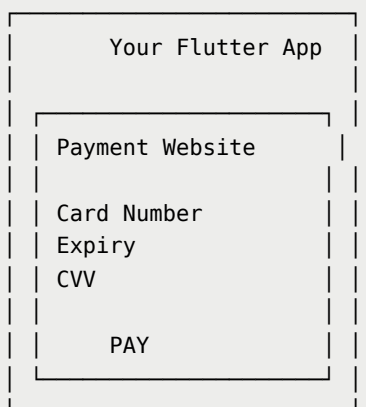
WebView approach

Now imagine the payment provider gives you a **web payment page**.

Your Flutter application opens that webpage inside a WebView.



The user sees something like:



Your application doesn't necessarily build the payment form itself.

The provider hosts it.

Advantage

Easy integration and provider-controlled payment UI.

Your app can avoid directly handling sensitive payment-form data, depending on the provider's flow.

Disadvantage

You have less control over:

- UI
- user experience
- navigation
- web compatibility
- communication between WebView and Flutter

And you have to handle things like:

```
WebView
↓
redirect
↓
callback/deep link
↓
Flutter
```

API vs SDK vs WebView

	API	SDK	WebView
What is it?	Communication interface	Development toolkit	Embedded web page
Control	High	Medium	Low
Development effort	High	Low/Medium	Low/Medium
UI control	High	Depends on SDK	Low
Provider handles implementation?	Low	High	High

	API	SDK	WebView
Native integration	You implement	SDK may provide	Mostly Web
Dependency	API specification	SDK version/provider	Web page/provider
Typical use	Custom integration	Native/Flutter payment integration	Hosted checkout

Same payment system - three approaches

Imagine a provider called:

PayFast

API

You do:

```
Flutter
  ↓
HTTP POST
  ↓
PayFast API
```

You build the payment flow yourself.

SDK

You do:

```
Flutter
  ↓
PayFast SDK
  ↓
PayFast API
```

The SDK abstracts much of the API interaction.

WebView

You do:

```
Flutter
  ↓
WebView
  ↓
https://payfast.com/checkout
```

The provider's website handles the checkout UI.

The key technical difference

Think about **abstraction level**:

```
LESS CONTROL
  ↑
WebView
  ↑
SDK
  ↑
API
  ↓
MORE CONTROL
```

But there's an important nuance:

SDK and API aren't opposites.

An SDK often **uses the API internally**.

```
YOUR APP
  ↓
SDK
  ↓
HTTP/API calls
  ↓
Payment Provider
```

So an SDK is often an **abstraction layer over APIs**, plus additional tools/features.

Why would a developer choose API?

Suppose you have a complex fintech application and want your own payment UI.

API gives you control over:

```
UI
↓
Business logic
↓
API request
↓
Payment provider
```

You can design the architecture exactly how you want.

Main technical advantage:

Maximum flexibility and control.

Why choose SDK?

Suppose the provider offers:

```
Flutter SDK
```

Instead of spending days implementing:

```
HTTP
authentication
serialization
callbacks
error handling
platform integration
```

you might get:

```
PaymentSDK.pay(...);
```

Main technical advantage:

Reduced development effort and provider-supported integration.

Why choose WebView?

Suppose the provider says:

"We don't have a Flutter SDK, but we provide a hosted checkout page."

You can use:

```
Flutter
  ↓
WebView
  ↓
Hosted Checkout
```

Main technical advantage:

You can integrate a provider's existing web payment flow without implementing the entire payment UI natively.

Question:

You're developing a Flutter app that needs an external payment system. Compare API, SDK and WebView.

A strong answer:

API: Provides direct communication with the payment service. It offers the greatest control and flexibility but requires more implementation work.

SDK: Provides prebuilt libraries, methods, and tools that abstract API communication and platform-specific details. It reduces development effort but introduces dependency on the provider's SDK.

WebView: Embeds the provider's web-based payment page inside the application. It is useful when a hosted checkout is available, but provides less control over the UI and requires handling redirects/callbacks.

Memorize:

```
API      → "I build the integration."  
SDK      → "Provider gives me tools to build it."  
WebView → "Provider gives me the web payment page."
```

What is a Widget?

A **Widget** is a building block of a Flutter UI.

Examples:

```
Text() Container() Row() Column() Image() Button
```

A Flutter app is basically a **tree of widgets**:

```
MaterialApp  
  ↓  
  Scaffold  
    ↓  
    Column  
    ├── Text  
    ├── Image  
    └── Button
```

Two main types

Flutter mainly has:

```
Widget  
  |  
  ├── StatelessWidget  
  └── StatefulWidget
```

StatelessWidget

A `StatelessWidget` is a widget whose UI **does not have mutable state of its own**. Once built, it displays UI based on the values given to it.

Example

```
class MyText extends StatelessWidget {  
  final String name;  
  
  const MyText({super.key, required this.name});  
  
  @override  
  Widget build(BuildContext context) {  
    return Text(name);  
  }  
}
```

Use it:

```
MyText(name: "Kinza")
```

The widget displays:

```
Kinza
```

Important:

```
StatelessWidget  
  ↓  
  build()  
  ↓  
  UI
```

If you want different data, you generally provide different values and Flutter rebuilds the widget.

StatefulWidget

A `StatefulWidget` is used when the UI needs to **change during the lifetime of the widget** because of mutable state.

Examples:

- Counter
- Checkbox
- Switch
- Text field
- Loading indicator
- Animation
- Selected tab
- Form state

Classic counter example

```
class Counter extends StatefulWidget {
  const Counter({super.key});

  @override
  State<Counter> createState() => _CounterState();
}

class _CounterState extends State<Counter> {
  int count = 0;

  void increment() {
    setState(() {
      count++;
    });
  }
}
```

```

@override
Widget build(BuildContext context) {
  return Column(
    children: [
      Text("$count"),
      ElevatedButton(
        onPressed: increment,
        child: const Text("Increase"),
      ),
    ],
  );
}

```

When the button is clicked:

```

count = 0
  ↓
button clicked
  ↓
setState()
  ↓
count = 1
  ↓
build() runs again
  ↓
UI shows 1

```

Why do we need `setState()` ?

This is **very important for exams**.

If you simply do:

```
count++;
```

Flutter doesn't automatically know that it needs to rebuild the UI.

Instead:

```
setState(() {count++;  
});
```

`setState()` tells Flutter:

"My state has changed; rebuild this widget's UI."

Biggest difference

StatelessWidget	StatefulWidget
No mutable state of its own	Has mutable state
UI generally doesn't change due to internal state	UI can change
Usually simpler	More complex
Only <code>build()</code> in the widget class	Widget + separate <code>State</code> class
Doesn't use <code>setState()</code>	Can use <code>setState()</code>
Good for static/presentational UI	Good for dynamic UI

Very important: StatefulWidget is actually TWO objects/classes

This confuses beginners.

When you write:

```
classCounterextendsStatefulWidget
```

you also create:

```
class_CounterStateextendsState<Counter>
```

Structure:

```
Counter
(StatefulWidget)
  |
  ↓
  _CounterState
  (State)
    |
    ↓
    build()
    |
    ↓
    UI
```

The **StatefulWidget** itself is immutable.

The mutable data is stored in the **State** object.

Lifecycle of StatefulWidget

Your instructor may ask this next.

Basic lifecycle:

```
createState()
  ↓
  initState()
  ↓
  build()
  ↓
  setState()
  ↓
  build()
  ↓
  dispose()
```

Important methods:

initState()

Called when the State object is initialized.

```
@override void initState() {super.initState();
```

```
}
```

Used for one-time initialization.

build()

Builds the UI.

```
@overrideWidgetbuild(BuildContextcontext) {returnText("Hello");  
}
```

setState()

Notifies Flutter that state changed.

```
setState(() {count++;  
});
```

dispose()

Called when the State object is permanently removed.

```
@overridevoiddispose() {controller.dispose();super.dispose();  
}
```

Used for cleanup.

One-line answer:

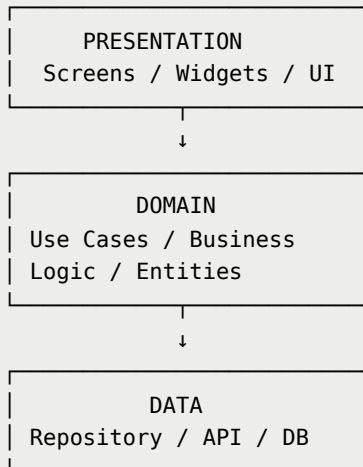
StatelessWidget is used when a widget does not manage changing internal state, while **StatefulWidget** is used when the UI depends on mutable state that can change during the widget's lifetime.

And the **big trick**:

`StatefulWidget` itself is immutable; its associated `State` object holds the mutable state.

Flutter app architecture: 3 layers

A common architecture is:



Think:

Presentation

What the user sees and interacts with.

Examples:

```
LoginScreen  
HomeScreen  
Button  
TextField  
Loading indicator
```

Domain

What the application needs to do.

Examples:

```
LoginUser  
GetProducts  
MakePayment  
CalculateTotal
```

Data

Where the data actually comes from.

Examples:

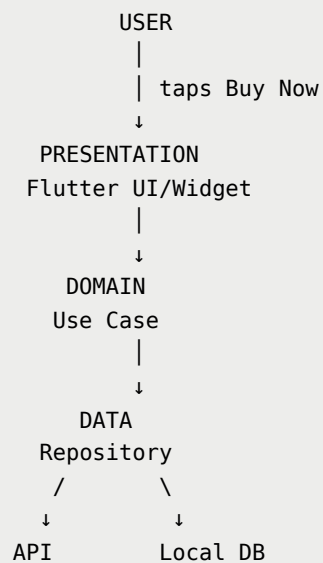
```
REST API  
Firebase  
SQLite  
Local storage  
Database
```

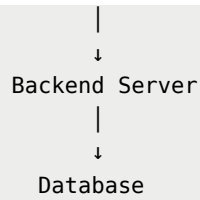
Complete user → database → user flow

Imagine the user opens a shopping app and presses:

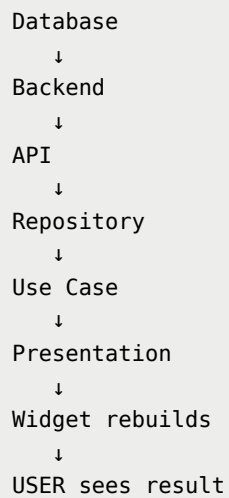
| **"Buy Now"**

The sequence is roughly:

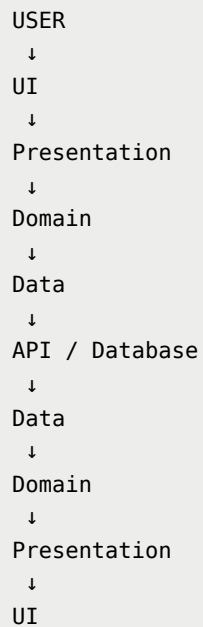




Then the result comes back:



So the overall flow is:



↓
USER

Let's take a real Flutter example

Suppose the user opens a **Product Screen**.

Step 1 - User

User taps:

"View Products"

↓

Step 2 - Presentation

Flutter widget receives the interaction:

```
ElevatedButton(  
  onPressed: () {  
    // trigger action  
  },  
  child: Text("View Products"),  
)
```

The UI shouldn't ideally contain all the business logic.

Instead, it triggers something in the presentation layer/state management.

Presentation → Domain

Presentation says:

| "I need products."

The domain has a use case:

```
class GetProducts {
  final ProductRepository repository;

  GetProducts(this.repository);

  Future<List<Product>> execute() {
    return repository.getProducts();
  }
}
```

The domain doesn't care whether the products come from:

```
REST API
Firebase
SQLite
Mock data
```

It just says:

| Give me products.

Domain → Data

The domain calls the repository:

```
GetProducts
  ↓
ProductRepository
```

The repository belongs to the data side.

For example:

```
class ProductRepository {
  Future<List<Product>> getProducts() async {
    // get data from API
  }
}
```

```
}  
}
```

Data → API

Repository may call:

```
ProductRepository  
  ↓  
  HTTP/API  
  ↓  
Backend Server
```

For example:

```
GET /products
```

The backend receives the request.

Backend → Database

Now the backend might query the database:

```
Backend  
  ↓  
Database  
  ↓  
Products table
```

For example:

```
SELECT*FROM products;
```

The database returns:

Laptop
Phone
Headphones

Now everything comes back

This is VERY important.

The flow is not only:

Flutter → DB

There is a **return path**.

```
Database
  ↓
Backend
  ↓
API response
  ↓
Repository
  ↓
Domain/use case
  ↓
Presentation/state
  ↓
Widget
  ↓
User
```

Eventually the UI displays:

Laptop
Phone
Headphones

Where does Flutter's `build()` fit?

Now connect this with your previous question.

Suppose the screen initially has:

```
Loading...
```

Then the API returns products.

The state changes:

```
setState(() {products=response;isLoading=false;
});
```

Flutter rebuilds the relevant UI:

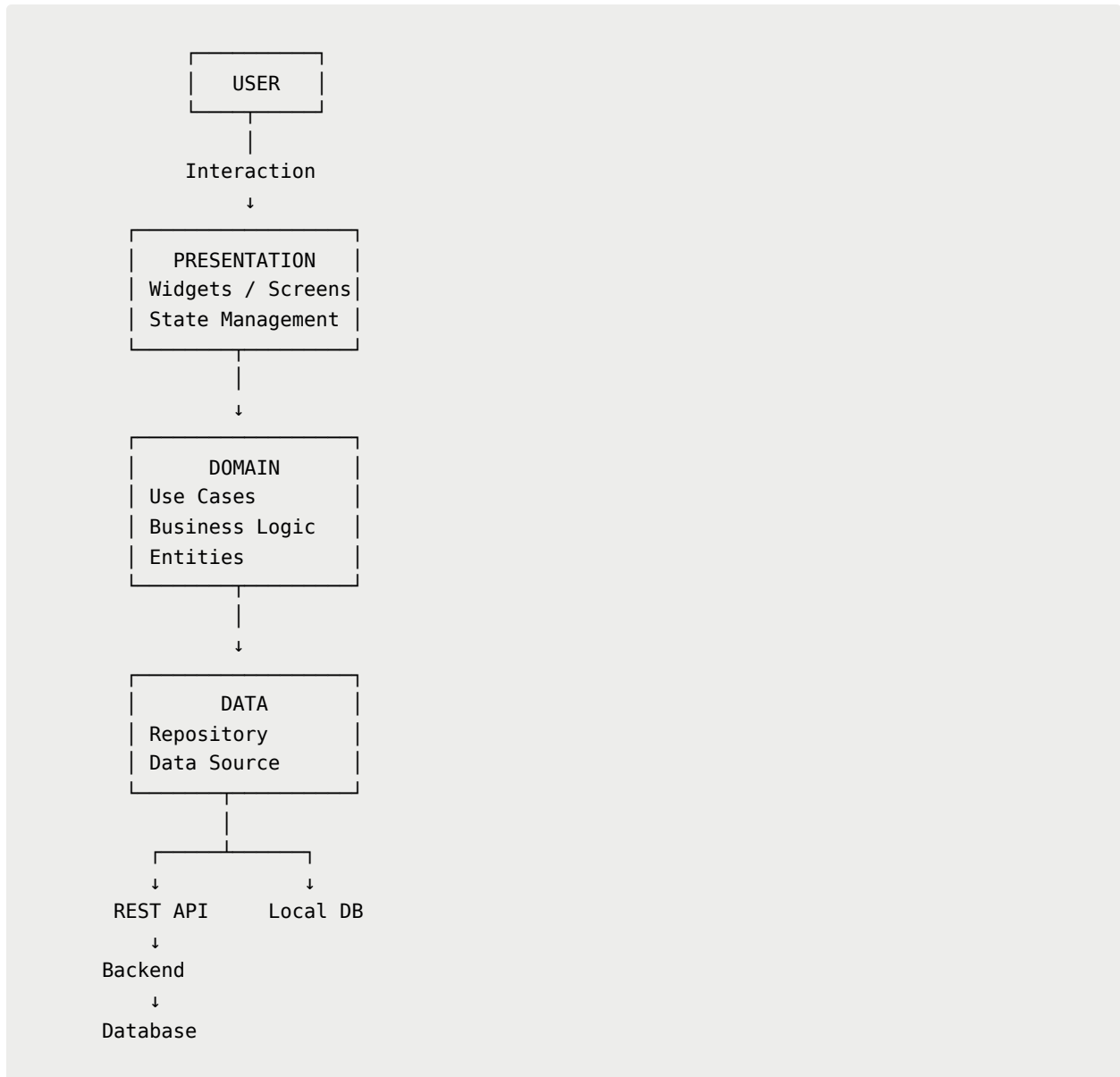
```
setState()
  ↓
build()
  ↓
New UI
  ↓
User sees products
```

So:

```
API response
  ↓
Update state
  ↓
setState()
  ↓
build()
  ↓
UI changes
```

Complete sequence for your exam

You can draw this:



And the response travels **back upward**.

If your instructor asks:

"Explain the flow of data in a Flutter application using presentation, domain and data layers."

Say:

"The user interacts with the presentation layer, which contains the UI and handles user actions. The presentation layer invokes the appropriate use case in the domain layer, where business logic is performed. The domain layer communicates with the data layer through repositories. The data layer retrieves or modifies data using APIs, databases, or local data sources. The result then travels back through the data, domain, and presentation layers, causing the UI to update and displaying the result to the user."

A repository is an abstraction layer that manages access to data sources such as APIs, databases, or local storage and provides the required data to the domain layer.

Flutter `ListView` and `ListTile`

These are **very common Flutter widgets for displaying lists**.

1. `ListView`

`ListView` is a scrollable list of widgets.

```
ListView(  
  children: [  
    Text("Apple"),  
    Text("Banana"),  
    Text("Mango"),  
  ],  
)
```

If there are many items, `ListView` allows the user to **scroll** through them.

2. `ListTile`

`ListTile` is a pre-designed row commonly used **inside a `ListView`**.

It provides areas like:

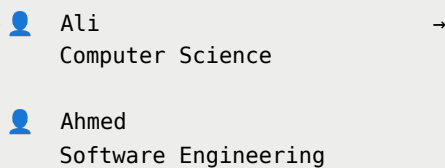
- `leading` → widget at the start
- `title` → main text

- `subtitle` → smaller text below
- `trailing` → widget at the end
- `onTap` → action when tapped

Example:

```
ListView(  
  children: [  
    ListTile(  
      leading: Icon(Icons.person),  
      title: Text("Ali"),  
      subtitle: Text("Computer Science"),  
      trailing: Icon(Icons.arrow_forward),  
      onTap: () {  
        print("Ali clicked");  
      },  
    ),  
    ListTile(  
      leading: Icon(Icons.person),  
      title: Text("Ahmed"),  
      subtitle: Text("Software Engineering"),  
    ),  
  ],  
)
```

Visually:



ListView vs **ListTile**

<code>ListView</code>	<code>ListTile</code>
Creates a scrollable list	Creates a single list row
Container for multiple items	One item/row
Can contain any widgets	Has predefined row structure
Handles scrolling	Does not provide scrolling itself

Think:

```

ListView
|
├─ ListTile
├─ ListTile
├─ ListTile
└─ ListTile

```

Exam point

ListView = list/container that scrolls

ListTile = individual formatted row inside the list

3. `ListView.builder()` - important

For a **large/dynamic list**, use `ListView.builder()`.

```

List<String> names = ["Ali", "Ahmed", "Sara", "Ayesha"];

ListView.builder(
  itemCount: names.length,
  itemBuilder: (context, index) {
    return ListTile(
      leading: Icon(Icons.person),
      title: Text(names[index]),
    );
  },
)

```

Here:

```
itemCount → number of items  
index      → current item's index  
itemBuilder → builds each item
```

Why **builder** ?

Instead of creating every item at once, Flutter builds items **as needed while scrolling**, which is more suitable for large lists.

Memory trick:

ListView → manually give children

ListView.builder → generate children dynamically

ListTile → design one row

FormData / Multipart Form Data in Flutter

Multipart form data is used when you need to send **files + normal form fields** to an API.

For example, uploading a profile picture:

```
Flutter App  
  ↓  
Multipart Request  
  ├── name = Kinza  
  ├── email = kinza@gmail.com  
  └── image = profile.jpg  
  ↓  
Backend API
```

Why "multipart"?

Normal form data sends text values:

```
name=Kinza  
email=kinza@gmail.com
```

But a file is binary data. `multipart/form-data` allows you to send different types of data together in one HTTP request.

Flutter example using `http`

```
import 'package:http/http.dart' as http;
import 'dart:io';

Future<void> uploadImage(File image) async {
  var request = http.MultipartRequest(
    'POST',
    Uri.parse('https://example.com/upload'),
  );

  // Normal form fields
  request.fields['name'] = 'Kinza';
  request.fields['email'] = 'kinza@gmail.com';

  // File
  request.files.add(
    await http.MultipartFile.fromPath(
      'image',
      image.path,
    ),
  );

  var response = await request.send();

  print(response.statusCode);
}
```

Important parts

```
request.fields['name']='Kinza';
```

→ Sends a **normal text field**.

```
request.files.add(  
    await http.MultipartFile.fromPath('image', image.path),  
);
```

→ Sends a **file**.

```
request.send();
```

→ Sends the multipart request.

Exam definition

Multipart form data is an HTTP request format used to send multiple types of data, especially text fields and files, in a single request.

Common use cases

- Profile picture upload
- PDF/document upload
- Multiple image upload
- Registration form + image
- Product creation + product images

Common trap

multipart/form-data ≠ JSON

JSON:

```
{  
  "name": "Kinza",  
  "email": "kinza@gmail.com"  
}
```

Multipart:

```
name → Kinza  
email → kinza@gmail.com  
image → profile.jpg
```

So remember:

JSON → structured data

Multipart → fields + files

Serialization, Deserialization, `toJson()` and `fromJson()` in Flutter

This is basically about **converting Dart objects** ↔ **JSON** when communicating with an API.

1. Serialization

Dart Object → **JSON**

You have a Dart object and want to send it to an API.

```
Dart Object  
  ↓  
Serialization  
  ↓  
JSON
```

Usually done with:

```
toJson()
```

Example:

```
class User {  
  String name;  
  int age;
```

```

    User(this.name, this.age);

    Map<String, dynamic> toJson() {
      return {
        'name': name,
        'age': age,
      };
    }
  }
}

```

Usage:

```

User user = User("Kinza", 21);

print(user.toJson());

```

Output:

```
{name: Kinza, age: 21}
```

If sending through an HTTP API:

```
jsonEncode(user.toJson());
```

Output:

```

{
  "name": "Kinza",
  "age": 21
}

```

2. Deserialization

JSON → Dart Object

When the API sends JSON back to your Flutter app, you convert it into a Dart object.

```
JSON
  ↓
Deserialization
  ↓
Dart Object
```

Usually done with:

```
fromJson()
```

Example:

```
class User {
  String name;
  int age;

  User(this.name, this.age);

  factory User.fromJson(Map<String, dynamic> json) {
    return User(
      json['name'],
      json['age'],
    );
  }
}
```

Usage:

```
Map<String, dynamic> data = {
  'name': 'Kinza',
  'age': 21,
```

```
};

User user = User.fromJson(data);

print(user.name);
print(user.age);
```

Output:

```
Kinza
21
```

toJson() VS fromJson()

Method	Direction	Purpose
toJson()	Dart → JSON	Send data to API
fromJson()	JSON → Dart	Receive data from API

Memory trick:

```
TO JSON
Dart → JSON

FROM JSON
JSON → Dart
```

Complete API example

Suppose your backend returns:

```
{
  "id":10,
  "name":"Kinza",
```

```
"email": "kinza@gmail.com"  
}
```

Your Dart model:

```
class User {  
  int id;  
  String name;  
  String email;  
  
  User({  
    required this.id,  
    required this.name,  
    required this.email,  
  });  
  
  // JSON → Dart  
  factory User.fromJson(Map<String, dynamic> json) {  
    return User(  
      id: json['id'],  
      name: json['name'],  
      email: json['email'],  
    );  
  }  
  
  // Dart → JSON  
  Map<String, dynamic> toJson() {  
    return {  
      'id': id,  
      'name': name,  
      'email': email,  
    };  
  }  
}
```

Receiving from API

```
final jsonData = jsonDecode(response.body);  
  
User user = User.fromJson(jsonData);
```

Now:

```
print(user.name);
```

gives:

```
Kinza
```

Sending to API

```
User user = User(  
  id: 10,  
  name: "Kinza",  
  email: "kinza@gmail.com",  
);  
  
final jsonData = jsonEncode(user.toJson());
```

What is "form parsing"?

Don't confuse **form parsing** with JSON serialization.

If the user enters:

```
Name: Kinza  
Age: 21
```

Flutter gets values from form fields, e.g.:

```
TextEditingController nameController =  
    TextEditingController();  
  
String name = nameController.text;
```

You can then put those values into your model:

```
User user = User(  
    nameController.text,  
    int.parse(ageController.text),  
);
```

Then:

```
user.toJson()
```

converts the model into data suitable for an API.

Overall flow

```
USER FORM  
  ↓  
TextEditingController  
  ↓  
Dart Model/Object  
  ↓  
toJson()  
  ↓  
JSON  
  ↓  
API  
  ↓  
JSON Response  
  ↓  
fromJson()  
  ↓  
Dart Model/Object  
  ↓  
Flutter UI
```

Exam answer

Serialization converts a Dart object into JSON, commonly using `toJson()`.

Deserialization converts JSON into a Dart object, commonly using `fromJson()`.

This is commonly used when sending and receiving data through REST APIs.